

Eqntott: Truth Table Generator from Boolean Equations

This tutorial demonstrates the use of eqntott software with some code examples

SHAKTHI KANNAN

eqntott (short for 'equation to truth table') is a software tool that can generate truth tables from Boolean equations, which can be used for programmable logic array (PLA) programming. It was initially written at Berkeley, and ported to work on GNU/Linux. It is released under the new BSD licence.

Consider a simple AND example:

```
NAME = and;
INORDER = a b;
OUTORDER = c;
c = a & b;
```

Here NAME refers to the name of the PLA, and is called 'and'. INORDER lists the input order of elements, while OUTORDER has the list of outputs in the truth table. Copy this example in a text editor and save it as 'and.eqn'.

To run the above example, use 'cd' command in order to go to the directory where the example is saved. Now run the following command:

```
$ eqntott and.eqn
```

You will get the output as:

```
.i 2
.o 1
.p 1
11 1
.e
```

'i' in the output refers to the number of inputs defined, which refers to 'a' and 'b' in the above example. 'o' corresponds to the number of output variables, which is 'c'. 'p' is the number of product terms. The truth table generated shows that the output is '1' when both the inputs are '1'.

You can use '-f' option with eqntott

to output the PLA name, input and output elements:

```
$ eqntott -l and.eqn
```

The output will be:

```
.i 2
.o 1
.na and
.ilb a b
.ob c
.p 1
11 1
.e
```

The name of the PLA is mentioned next to '.na' in the output. The order of inputs and outputs in the truth table is also listed.

The following expressions are allowed in the equations for eqntott:

Expression	Meaning
&	Logical AND operation
	Logical OR operation
!	Logical NOT operation
ZERO or 0	False or the value zero
ONE or 1	True or the value one
()	To enclose any expression
?	Don't care condition
name	Any input/output in the expression

The half adder circuit adds two binary digits, and produces a sum and a carry. It can be implemented as follows:

```
NAME = half_adder;
INORDER = a b;
OUTORDER = s c;
c = a & b;
s = (!a & b) | (a & !b);
```

When the above equation is run with eqntott:

```
$ eqntott -l half_adder.eqn
```

The output will be:

```
.i 2
.o 2
.na half_adder
.ilb a b
.ob s c
.p 3
01 1 0
10 1 0
11 0 1
.e
```

The sum is represented by 's' and carry with 'c'. When either the sum or carry is '1', the sum is '1' and carry is '0'. When both the sum and carry are '1', the sum is '0' and carry is '1'.

The output of eqntott can be customised using '-key' argument. The default option is the string 'iopte'. A few key code options with their output meaning are given below:

Character	Output
e	.e
f	.f output-number input-number
h	Human readable format
i	.i number of inputs
l	Truth table with PLA name, inputs and outputs
o	.o number of outputs
p	.p number-of-product-terms
v	eqntott version number
S	PLA connectivity summary

If the half adder example is run with the following key options:

```
$ eqntott -f,iopte half_adder.eqn
```

The output will be:

```
.na half_adder
.ilb a b
.ob s c
```